

ΚΕΦΑΛΑΙΟ 8

Τελική Επανάληψη Fortran

Σύντομες βασικές σημειώσεις για ασκήσεις και εργαστήρια

Σκοπός αυτού του κεφαλαίου δεν είναι να ξαναδιδάξει όλη τη θεωρία. Είναι να λειτουργεί ως γρήγορος οδηγός πριν λύσεις άσκηση, εργαστήριο ή θέμα: τι αναγνωρίζω στην εκφώνηση, ποιο μοτίβο επιλέγω και τι ελέγχω πριν παραδώσω.

Χρειάζεσαι να θυμηθείς...	Πήγαινε στην ενότητα
σκελετό, τύπους και εκφράσεις	8.2
IF και επιλογή περιπτώσεων	8.3
DO, μέτρηση, άθροισμα, τερματισμό	8.4
πίνακες και RESHAPE	8.5
FUNCTION, SUBROUTINE, MODULE	8.6
αρχεία και δεδομένα αγνώστου πλήθους	8.7
PRINT, WRITE και FORMAT	8.8
έτοιμα μοτίβα εργαστηρίων	8.9

Κεντρικός κανόνας: πρώτα σχεδιάζω τη λύση με λόγια και μετά γράφω Fortran. Έτσι επιλέγω σωστά μεταβλητές, βρόχους, πίνακες και διαδικασίες.

8.1 Από την εκφώνηση στον κώδικα

Κάθε άσκηση, μικρή ή μεγάλη, λύνεται με την ίδια σειρά. Μην ξεκινάς γράφοντας DO ή IF πριν ξέρεις τι πρέπει να υπολογιστεί.

Βήμα	Ερώτηση που απαντώ	Συνηθισμένο αποτέλεσμα
1. Δεδομένα	Τι δίνεται; μία τιμή, N τιμές, πίνακας ή αρχείο;	μεταβλητές / arrays / READ
2. Ζητούμενα	Τι πρέπει να βρω ή να εκτυπώσω;	άθροισμα, μέσος, πλήθος, πίνακας, αρχείο
3. Απόφαση	Υπάρχει όριο ή διαφορετική περίπτωση;	IF / ELSE IF
4. Επανάληψη	Επαναλαμβάνω για N στοιχεία ή μέχρι συνθήκη;	DO / DO WHILE / EXIT
5. Οργάνωση	Επαναχρησιμοποιώ καθαρό υπολογισμό;	FUNCTION / SUBROUTINE / MODULE
6. Έλεγχος	Ταιριάζει ο τύπος; αρχικοποίησα; χωρά ο format;	δοκιμή με μικρές τιμές

Μικρό παράδειγμα σχεδίασης

«Διάβασε N θερμοκρασίες και βρες τον μέσο όρο των θετικών.» Άρα: N και i είναι INTEGER, κάθε τιμή και το sum είναι REAL, χρειαζομαι DO 1..N, IF t>0, έναν μετρητή count και στο τέλος έλεγχο count>0 πριν τη διαίρεση.

```
sum_t = 0.0
count = 0
do i = 1, n
  read(*,*) t
  if (t > 0.0) then
    sum_t = sum_t + t
    count = count + 1
  end if
end do
if (count > 0) print *, sum_t / real(count)
```

Αυτό είναι το πιο συχνό μοτίβο των ασκήσεων: αρχικοποίηση -> βρόχος -> προαιρετικό IF -> τελικό αποτέλεσμα.

8.2 Σκελετός προγράμματος, τύποι και εκφράσεις

```
program main
implicit none
integer :: i, n, count
real :: x, sum_x, average
logical :: ok
```

! δηλώσεις πρώτα - εκτελέσιμες εντολές μετά

```
end program main
```

Τύπος	Χρήση	Παραδείγματα
INTEGER	μετρήσεις, δείκτες, πλήθος	i, n, count
REAL	δεκαδικές τιμές, μέσοι όροι	t, sum_t, average
LOGICAL	σωστό/λάθος	found, ok
CHARACTER	κείμενο	name, title

Τρία σημεία που εμφανίζονται διαρκώς

Γραφή	Θυμήσου
$x = x + 1$	Δεν είναι εξίσωση. Πάρε την παλιά τιμή του x και αύξησέ την κατά 1.
$\text{real}(a) / \text{real}(b)$	Μετατροπή σε REAL, ώστε να προκύψει πραγματική και όχι ακέραια διαίρεση.
**	Δύναμη: x^{**2} σημαίνει x στο τετράγωνο.
abs, sqrt, mod, min, max	Εγγενείς συναρτήσεις για συχνές πράξεις. Προσοχή στον τύπο των ορισμάτων.

Συχνό λάθος: $5/2$ δίνει 2 επειδή και οι δύο τιμές είναι INTEGER. Για 2.5 γράφουμε $5.0/2.0$ ή $\text{real}(5)/2.0$.

Ονόματα: αρχίζουν με γράμμα, μπορούν να έχουν γράμματα, αριθμούς και `_`, και η Fortran δεν ξεχωρίζει κεφαλαία/πεζά. Διάλεξε περιγραφικά ονόματα: temperature αντί για x όταν η έννοια είναι θερμοκρασία.

8.3 Συνθήκες: IF, ELSE IF, ELSE

```
if (temperature >= 22.0 .and. temperature <= 25.5) then
  print *, 'άνετη θερμοκρασία'
else if (temperature < 22.0) then
  print *, 'κρύο'
else
  print *, 'ζέστη'
end if
```

Χρειάζεσαι...	Χρησιμοποίησε
σύγκριση	==, /=, <, <=, >, >=
και οι δύο συνθήκες αληθείς	.and.
τουλάχιστον μία αληθής	.or.
άρνηση	.not.
μία διαδρομή από πολλές	IF - ELSE IF - ELSE

Σκέψη για όρια

Για «από 40 έως 60» πρέπει να ισχύουν και τα δύο: $x \geq 40$.and. $x \leq 60$. Για «έξω από το διάστημα» μπορούμε να γράψουμε $x < 40$.or. $x > 60$.

```
! μέτρηση τιμών που ικανοποιούν κριτήριο
if (humidity >= 40.0 .and. humidity <= 60.0 .and. &
  temperature >= 22.0 .and. temperature <= 25.5) then
  n_comfort = n_comfort + 1
end if
```

Πριν από κάθε IF: γράψε με λόγια τη συνθήκη. Μετά μετάφρασε τη σε συγκρίσεις και λογικούς τελεστές. Είναι ο ασφαλέστερος τρόπος να μην μπερδευτούν τα .and. και .or.

8.4 Επαναλήψεις: το βασικό εργαλείο των εργαστηρίων

Περίπτωση	Δομή	Θυμήσου
γνωστό πλήθος επαναλήψεων	do i = 1, n	Χρησιμοποίησε i ως δείκτη ή μετρητή.
συνθήκη πριν από κάθε γύρο	do while (condition)	Πρόσεξε να αλλάζει κάτι ώστε να τερματίσει.
άγνωστο πλήθος από αρχείο	do ... if (ierr /= 0) exit	Ο έλεγχος γίνεται αμέσως μετά τη READ.
παράλειψη μόνο του τρέχοντος γύρου	cycle	Συνέχισε στον επόμενο γύρο.
οριστική έξοδος	exit	Βγες από τον βρόχο.

Συσσωρευτές και μετρητές

```
sum_x = 0.0 ! πριν από το DO
product = 1.0 ! για γινόμενο
count = 0 ! για πλήθος
```

```
do i = 1, n
  sum_x = sum_x + a(i)
  if (a(i) > limit) count = count + 1
end do
average = sum_x / real(n)
```

Το αρχικό 0 για άθροισμα και 1 για γινόμενο είναι απαραίτητο. Αν ξεχάσεις αρχικοποίηση, το πρόγραμμα ξεκινά από άγνωστη τιμή.

```
! ασφαλής τερματισμός όταν δεν ξέρω πόσες γραμμές θα διαβαστούν
do
  read(u,*,iostat=ierr) value
  if (ierr /= 0) exit
! επεξεργασία της value μόνο μετά από επιτυχή READ
end do
```

8.5 Συστοιχίες: πολλά δεδομένα με ένα όνομα

```
integer :: i
real :: a(5), x(2,3)

a = [1.0, 2.0, 3.0, 4.0, 5.0]
print *, a(3) ! το τρίτο στοιχείο
print *, a(2:4) ! τμήμα: δεύτερο έως τέταρτο
```

Χρειάζεσαι...	Σύνταξη / ιδέα
μονοδιάστατο array	real :: a(n) ή real :: a(5)
δισδιάστατο array	real :: x(rows, columns)
πλήθος στοιχείων	size(a)
διαστάσεις	shape(x)
άθροισμα / ελάχιστο / μέγιστο	sum(a), minval(a), maxval(a)
στοιχείο προς στοιχείο	c = a + b, αν έχουν ίδια shape

RESHAPE και σειρά γέμισης

```
integer :: x(6), y(2,3), z(3,2)
x = [1, 2, 3, 4, 5, 6]
y = reshape(x, [2,3])
z = reshape(x, [3,2])
```

Η Fortran γεμίζει κατά στήλες. Για y(2,3): η πρώτη στήλη γίνεται 1,2, η δεύτερη 3,4 και η τρίτη 5,6. Γι' αυτό το ίδιο x δημιουργεί διαφορετική διάταξη στο z(3,2).

Πρότυπο επεξεργασίας array: διάβασε ή γέμισε στοιχεία μέσα σε DO, κάνε τον υπολογισμό, και εκτύπωσε με δεύτερο DO ή μορφοποιημένη WRITE.

8.6 Διαδικασίες: FUNCTION, SUBROUTINE και MODULE

Χρειάζομαι...	Επιλογή	Παράδειγμα χρήσης
ένα αποτέλεσμα	FUNCTION	average = mean(a)
να κάνω ενέργεια ή να επιστρέψω πολλά αποτελέσματα	SUBROUTINE	call analyse(a, mean, count)
πολλές σχετικές διαδικασίες	MODULE	use stats_mod, only: mean

```
module stats_mod
  implicit none
  contains
  real function mean(a)
    real, intent(in) :: a(:)
    mean = sum(a) / real(size(a))
  end function mean
end module stats_mod
```

```
program main
  use stats_mod, only: mean
  implicit none
  real :: a(3) = [2.0, 4.0, 6.0]
  print *, mean(a)
end program main
```

INTENT: intent(in) σημαίνει ότι η διαδικασία μόνο διαβάζει το όρισμα, intent(out) ότι επιστρέφει αποτέλεσμα και intent(inout) ότι το διαβάζει και το αλλάζει.

```
subroutine count_above(a, limit, count)
  implicit none
  real, intent(in) :: a(:), limit
  integer, intent(out) :: count
  integer :: i

  count = 0
  do i = 1, size(a)
    if (a(i) > limit) count = count + 1
  end do
end subroutine count_above
```

Σε SUBROUTINE πρέπει να δηλωθεί και ο δείκτης i: integer :: i. Κάθε ανεξάρτητη μονάδα χρειάζεται implicit none.

8.7 Αρχεία: OPEN - επεξεργασία - CLOSE

```
open(newunit=u, file='data.txt', status='old', action='read')
read(u,*) x
close(u)

open(newunit=v, file='results.txt', status='replace', action='write')
write(v,*) 'Αποτέλεσμα:', x
close(v)
```

Λέξη	Τι θυμάμαι
newunit=u	Η Fortran επιλέγει διαθέσιμο αριθμό μονάδας και τον αποθηκεύει στο u.
status='old'	Το αρχείο πρέπει να υπάρχει ήδη.
status='replace'	Δημιουργεί ή αντικαθιστά αρχείο αποτελεσμάτων.
action='read' / 'write'	Ανάγνωση ή εγγραφή.
iostat=ierr	Κωδικός κατάστασης. ierr==0 σημαίνει επιτυχία.
rewind(u)	Γυρίζει στην αρχή του ήδη ανοιχτού αρχείου.

Δεδομένα αγνώστου πλήθους: δύο περάσματα

```
real, allocatable :: h(:), t(:)

n = 0
do
  read(u,*,iostat=ierr) h_val, t_val
  if (ierr /= 0) exit
  n = n + 1
end do

rewind(u)
allocate(h(n), t(n))
do i = 1, n
  read(u,*) h(i), t(i)
end do
```

Η δήλωση `real, allocatable :: h(:), t(:)` λέει ότι οι `h` και `t` είναι δυναμικοί μονοδιάστατοι πίνακες, αλλά δεν τους δίνει ακόμη μέγεθος. Το `allocate(h(n),t(n))` δεσμεύει τις `n` θέσεις μόνο αφού βρούμε το `n`. Αν το μέγεθος ήταν γνωστό από την αρχή, θα μπορούσαμε να γράψουμε απλά `real :: h(100), t(100)`, χωρίς `ALLOCATE`. Στο τέλος: `deallocate(h, t)` και `close(u)`.

8.8 Μορφοποίηση: η ελάχιστη εργαλειοθήκη

Σύνταξη	Τι κάνει
print *, x	γρήγορη έξοδος στην οθόνη, ελεύθερη μορφή
read(*,*) x	είσοδος από πληκτρολόγιο, ελεύθερη μορφή
write(*,'(a,f8.2)') 'x = ', x	κείμενο και πραγματικός με 2 δεκαδικά
10 format(a,f7.5)	μορφότυπος σε ξεχωριστή πρόταση, που χρησιμοποιείται με print 10 ή write(*,10)
3(f8.2,1x)	τρία πραγματικά, καθένα με 2 δεκαδικά και ένα κενό
/ και 2X	νέα γραμμή και δύο κενές θέσεις
advance='no'	η WRITE δεν αλλάζει ακόμα γραμμή

```
character(len=12) :: name
integer :: n
logical :: found
real :: x
```

```
write(*,'(a12,1x,i4,1x,l2,1x,f8.2)') name, n, found, x
```

Σειρά κλειδί: A για κείμενο, I για INTEGER, L για LOGICAL, F για REAL. Η σειρά των περιγραφών πρέπει να είναι ακριβώς ίδια με τη σειρά των δεδομένων.

```
do i = 1, 5
  write(*,'(i4)',advance='no') i
end do
print *
```

Το advance='no' γράφεται στη WRITE, όχι μέσα στο FORMAT. Η τελική print * μεταφέρει τον δρομέα στην επόμενη γραμμή.

8.9 Πέντε μοτίβα που επανέρχονται στα εργαστήρια

Αν η εκφώνηση λέει...	Το μοτίβο είναι...	Κρίσιμη λεπτομέρεια
άθροισμα, παραγοντικό, σειρά	συσσωρευτής μέσα σε DO	sum=0, product=1 πριν από τον βρόχο
πλήθος τιμών που περνούν κριτήριο	DO + IF + count	count=0 πριν και count=count+1 μέσα στο IF
μέσος όρος επιλεγμένων τιμών	sum + count + τελική διαίρεση	έλεγχε count>0 πριν διαιρέσεις
N στοιχεία / matrix	array + DO ή array operations	δείκτες 1..N και συμβατό shape
κώδικας που επαναχρησιμοποιείται	FUNCTION ή SUBROUTINE	implicit none και σωστά INTENT
αρχείο άγνωστου πλήθους	IOSTAT -> count -> REWIND -> ALLOCATE -> ξαναδιάβασμα	μην χρησιμοποιήσεις τιμή όταν ierr/=0

Μοτίβο Lab: δεδομένα και κριτήριο άνεσης

```
if (h(i) >= 40.0 .and. h(i) <= 60.0 .and. &  
t(i) >= 22.0 .and. t(i) <= 25.5) then  
  n_comfort = n_comfort + 1  
  write(v,'(2(f10.2,1x))') h(i), t(i)  
end if  
  
percent = 100.0 * real(n_comfort) / real(n)
```

Το παράδειγμα συνδυάζει array, DO, IF, μετρητή, WRITE σε αρχείο και πραγματική διαίρεση. Είναι καλό μοντέλο για να συνδέσεις πολλές ενότητες σε μία άσκηση.

8.10 Τελικός έλεγχος πριν τρέξεις ή παραδώσεις

Έλεγχε	Ερώτηση
Δηλώσεις	Έχω implicit none; δήλωσα όλες τις μεταβλητές και δείκτες;
Τύπους	Είναι INTEGER ο δείκτης/μετρητής και REAL οι δεκαδικές τιμές;
Αρχικοποίηση	Έβαλα sum=0, count=0, product=1 πριν από το DO;
Όρια	Είναι σωστά τα <=, >= και το .and. / .or.;
Διάρθρωση	Υπάρχει περίπτωση διαίρεσης με μηδέν; χρειάζεται real(...);
Arrays	Είναι ο δείκτης εντός ορίων; έχουν τα arrays συμβατό shape;
Διαδικασίες	Έχω σωστά INTENT, ορίσματα και implicit none;
Αρχεία	Άνοιξα το σωστό όνομα, έλεγα ierr και έκλεισα το αρχείο;
Μορφοποίηση	Ταιριάζουν A/I/L/F με τα δεδομένα; χωρά το πλάτος F;
Δοκιμή	Έτρεξα με μικρές τιμές που μπορώ να υπολογίσω με το χέρι;

Μίνι checklist 60 δευτερολέπτων

- ☐ Καταλαβαίνω τι διαβάζω και τι ζητείται.
- ☐ Έχω επιλέξει σωστά IF, DO, array, διαδικασία ή αρχείο.
- ☐ Οι μεταβλητές έχουν σωστούς τύπους και αρχικές τιμές.
- ☐ Κάθε υπολογισμός γίνεται μέσα ή έξω από τον βρόχο στη σωστή θέση.
- ☐ Η έξοδος είναι καθαρή και περιλαμβάνει τα ζητούμενα.
- ☐ Το πρόγραμμα δοκιμάστηκε με ένα μικρό, ελεγχόμενο παράδειγμα.

Τελευταία υπενθύμιση: η σωστή λύση δεν είναι η πιο σύντομη. Είναι εκείνη που διαβάζεται εύκολα, ακολουθεί την εκφώνηση βήμα-βήμα και δίνει σωστό αποτέλεσμα για απλές δοκιμές πριν την παραδώσεις.